Design Patterns in C++

Named, proven solutions to recurring design problems. The five most common (Singleton, Factory, Strategy, Observer, State), how modern C++ changes them, and SOLID.

What patterns are and are not

A pattern is a **vocabulary**: "this is an Observer" conveys structure and intent. Three families:

FAMILY	CONCERNED WITH	EXAMPLES
Creational	how objects are created	Singleton, Factory, Builder
Structural	how objects are composed	Adapter, Decorator, Proxy, PIMPL
Behavioural	how objects interact and share responsibility	Strategy, Observer, State, Command, Visitor

NOT GOALS Each pattern adds indirection: use one only for a real problem, and know *when not to use it*.

MODERN C++ SHORTENS MANY A lambda or `std::function` replaces Strategy and Command hierarchies; `std::visit` replaces Visitor; smart pointers are a Proxy; RAII is built in.

SOLID principles

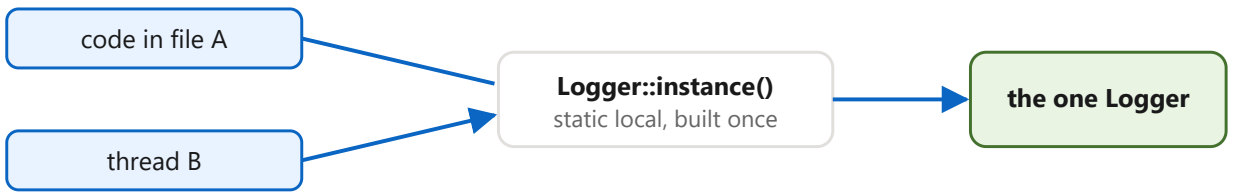
SOLID are five design principles. Most patterns in this lesson exist to serve one of them, so read this table first and refer back to it.

	PRINCIPLE	IN ONE SENTENCE	PATTERN THAT SERVES IT
S	Single Responsibility	a class has one reason to change	split parsing, logic and I/O into separate classes

O	Open/Closed	open for extension, closed for modification: add behaviour by adding code, not editing old code	Strategy, Factory registry, Decorator
L	Liskov Substitution	a derived object must be usable wherever the base is expected without breaking correctness	correct inheritance hierarchies
I	Interface Segregation	prefer several small interfaces over one fat one	narrow abstract classes
D	Dependency Inversion	depend on abstractions, not concrete classes; inject dependencies	Strategy, Observer, dependency injection (instead of Singleton)

Notes: Module 4, examples.

Singleton



Thread-safe creation since C++11; its methods still need their own locking.

Intent: exactly one instance with global access (logger, config).

```
class Logger {
public:
    static Logger& instance() {
        static Logger inst;              // created the first time control
passes here
        return inst;                     //  ...and the same object is
returned forever after
    }
    Logger(const Logger&) = delete;      // forbid copies: there must be only
one
    Logger& operator=(const Logger&) = delete;

    void log(const std::string& msg) { std::cout << msg << '\n'; }
private:
    Logger() { std::cout << "Logger constructed\n"; }   // private: nobody
else can create one
};

Logger::instance().log("hello");
```

The **Meyers Singleton**:

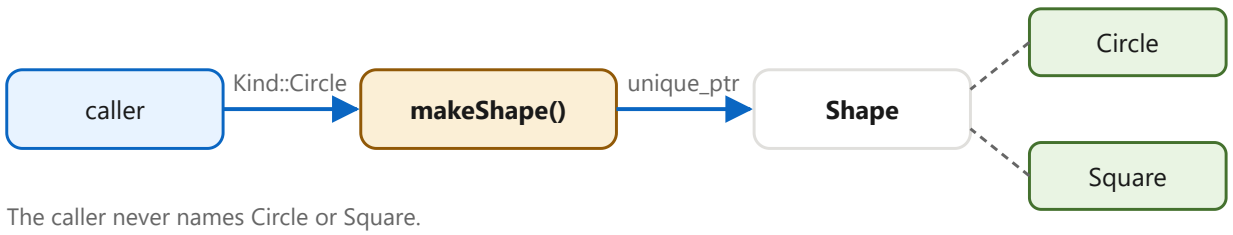
STEP THROUGH **Meyers Singleton: constructed once, shared forever**

```
1  static Logger& instance() {
2      static Logger inst;
3      return inst;
4  }
5
6  Logger& a = Logger::instance();
7  Logger& b = Logger::instance();
```

STEP	RUNS	WHAT HAPPENS	MEMORY AFTER (CHANGED IN BOLD)
1	line 6 `Logger& a = Logger::instance();`	First call to `instance()`. No Logger exists yet.	
2	line 2 `static Logger inst;`	Control reaches `static Logger inst;` for the **first time**, so the constructor runs now. The compiler guards this with a thread-safe flag. prints: `Logger constructed\n`	**`inst = constructed`**
3	line 3 `return inst;`	The function returns a **reference** to the static object. The reference **a** is bound to it.	`inst = constructed` | **`a = 0x4000`**
4	line 7 `Logger& b = Logger::instance();`	Second call. This time the flag says `inst` is already initialised, so the constructor line is **skipped**: nothing more is printed.	`inst = constructed` | `a = 0x4000`
5	line 7 `Logger& b = Logger::instance();`	**a** and **b** refer to the very same object (same address). That is the Singleton guarantee. Because the copy operations are deleted, you cannot accidentally make a second one with `Logger c = a;`.	`inst = constructed` | `a = 0x4000` | **`b = 0x4000`**

- **Thread safe creation:** a function-local `static` is initialized exactly once since C++11 ("magic statics"): no mutex or double-checked locking. Calls on the object still need their own locking.
- **Lazy:** built on first use, avoiding static initialization order problems.
- **Drawbacks (say them):** global state in disguise, hidden dependencies, hard to test with a fake, unspecified destruction order at exit.
- **Alternative:** create one object in `main` and pass it by reference (dependency injection).

Factory



The caller never names Circle or Square.

New type means editing the factory, unless it keeps a registry.

Intent: create objects without the caller naming the concrete class; it gets something usable only through an interface.

```
struct Shape {
    virtual double area() const = 0;
    virtual ~Shape() = default;
};
struct Circle : Shape { double r; explicit Circle(double r) : r(r) {} double
area() const override { return 3.14159 * r * r; } };
struct Square : Shape { double s; explicit Square(double s) : s(s) {} double
area() const override { return s * s; } };

enum class Kind { Circle, Square };

std::unique_ptr<Shape> makeShape(Kind k, double size) {       // the factory
function
    switch (k) {
        case Kind::Circle: return std::make_unique<Circle>(size);
        case Kind::Square: return std::make_unique<Square>(size);
    }
    return nullptr;
}

auto s = makeShape(Kind::Circle, 2.0);       // caller sees only Shape
std::cout << s->area();
```

- **Return** `unique_ptr`: clear ownership, no leaks, convertible to `shared_ptr`.
- The base needs a **virtual destructor**.
- A new type means editing the factory. Avoid with a *registry*: `map<string, function<unique_ptr<Shape>(double)>>` (Open/Closed).
- **Factory Method**: subclasses decide what to create. **Abstract Factory**: families of related objects (a matching `Button` and `Checkbox` per platform).

Strategy



The client picks the algorithm; modern C++ often uses a lambda instead of a class.

Intent: interchangeable algorithms chosen *at runtime* without changing the user (sort order, pricing, retry policy).

```
// 1. Classic: an interface and several implementations
struct PricingStrategy {
    virtual double apply(double price) const = 0;
    virtual ~PricingStrategy() = default;
};
struct Regular   : PricingStrategy { double apply(double p) const override {
return p; } };
struct Member    : PricingStrategy { double apply(double p) const override {
return p * 0.90; } };

class Checkout {
    std::unique_ptr<PricingStrategy> strategy_;
public:
    explicit Checkout(std::unique_ptr<PricingStrategy> s) :
strategy_(std::move(s)) {}
    double total(double price) const { return strategy_->apply(price); }
};

// 2. Modern: the strategy IS a callable. No class hierarchy needed.
class Checkout2 {
    std::function<double(double)> pricing_;
public:
    explicit Checkout2(std::function<double(double)> f) :
pricing_(std::move(f)) {}
    double total(double price) const { return pricing_(price); }
};
Checkout2 c([](double p) { return p * 0.70; });     // a lambda is a strategy

// 3. Compile-time: the strategy is a template parameter (policy-based
design): zero overhead
template <typename Pricing> class Checkout3 { /* ... Pricing{}(price) ... */
};
```

Try it:

TRY IT **Swap the strategy at runtime**

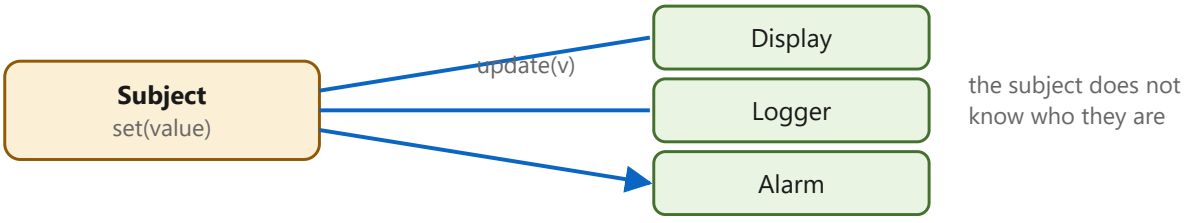
Strategy: Regular price | Price: 40

```
Checkout2 c([](double p) { return p; });
c.total(40);
```

Total: **40.00**. The `Checkout2` class is identical in every case; only the injected strategy changed.

STRATEGY VS STATE Similar shape, different intent. **Strategy**: the *client* picks the algorithm; strategies are independent. **State**: the object changes its own behaviour as its state changes, and states trigger the transitions.

Observer



Watch for dangling observers, and iterate over a copy while notifying.

Intent: a *subject* notifies its *observers* on change without knowing who they are (events, signals, pub/sub). Try it:

TRY IT **Subject: Thermometer**

Observers attached: ☑ Display ☑ Logger ☐ Alarm

set(25) Notify Clear log

```
(no output yet)
```

```
class Observer {
public:
    virtual void update(double value) = 0;
    virtual ~Observer() = default;
};

class Subject {
    std::vector<Observer*> observers_;
public:
    void attach(Observer* o) { observers_.push_back(o); }
    void detach(Observer* o) {
observers_.erase(std::remove(observers_.begin(), observers_.end(), o),
observers_.end()); }
    void set(double v) {
        value_ = v;
        auto copy = observers_;                    // iterate a COPY: an
observer may detach itself in update()
        for (auto* o : copy) o->update(v);
    }
private:
    double value_ = 0;
};

// Modern, lighter version: observers are just callbacks
std::vector<std::function<void(double)>> listeners;
listeners.push_back([](double t) { std::cout << "temp = " << t << '\n'; });
```

CLASSIC OBSERVER BUGS

Dangling observer: subject calls a destroyed object. Detach in the destructor (RAII) or store `weak_ptr`.

Modifying the list while notifying: iterate a copy.

Threads: lock, but never while calling observers (deadlock risk).

Step through a notification:

STEP THROUGH **Observer: notify everyone, even if one detaches itself**

```
1  Subject s;
2  s.attach(&display);
3  s.attach(&once);
4  s.set(21.5);
5  s.set(30);
```

STEP	RUNS	WHAT HAPPENS	MEMORY AFTER (CHANGED IN BOLD)
1	line 1 `Subject s;`	A subject with no observers yet.	**`observers = [ ]`**
2	line 2 `s.attach(&display);`	The Display attaches. The subject stores a pointer and knows nothing else about it.	**`observers = [ display ]`** | **`display = listening`**
3	line 3 `s.attach(&once);`	A **OneShot** observer attaches too. It will detach itself the first time it is notified.	**`observers = [ display, once ]`** | `display = listening` | **`once = listening`**
4	line 4 `s.set(21.5);`	`set(21.5)` first **copies** the list, then calls `update` on each. OneShot detaches while we are looping; because we iterate the copy, nothing is skipped or invalidated. prints: `Display shows 21.5\nOneShot saw 21.5 and detaches\n`	`observers = [ display, once ]` | **`copy = [ display, once ]`** | **`display = got 21.5`** | **`once = got 21.5`**
5	line 4 `s.set(21.5);`	After the notification the real list is **[display]**.	**`observers = [ display ]`** | **`display = listening`** ~~| once = detached~~
6	line 5 `s.set(30);`	Only Display is told about 30. Remaining hazards: an observer destroyed while still registered (dangling pointer: detach in the destructor or store `weak_ptr`), and notifying from several threads (lock, but not while calling out). prints: `Display shows 30\n`	`observers = [ display ]` | **`display = got 30`** ~~| once = detached~~

READ MORE

Effective Modern C++ (Meyers)

- Item 20, Use std::weak_ptr for shared_ptr-like pointers that can dangle (PDF pages 152 to 157)

State

Intent: behaviour changes with internal state, replacing scattered `if (state == ...)` checks (a turnstile). Send events:

TRY IT **Turnstile state machine**

current state
Locked

coin() push() Reset

STATE	COIN()	PUSH()
Locked	→ Unlocked (unlock the arm)	→ Locked (ALARM: pay first)
Unlocked	→ Unlocked (thanks, already open)	→ Locked (arm turns, then locks again)

```
start in state Locked
```

```cpp
// 1. Simplest: an enum and a switch. Fine for a few states.
enum class State { Locked, Unlocked };
class Turnstile {
    State s_ = State::Locked;
public:
    void coin() { s_ = State::Unlocked; }
    void push() { if (s_ == State::Unlocked) s_ = State::Locked; else
alarm(); }
};

// 2. Classic State pattern: one class per state
struct TurnstileState {
    virtual std::unique_ptr<TurnstileState> coin() = 0;     // each state
returns the NEXT state
    virtual std::unique_ptr<TurnstileState> push() = 0;
    virtual ~TurnstileState() = default;
};

// 3. Modern: std::variant of state types + std::visit; the compiler checks
every state is handled.
using StateV = std::variant<Locked, Unlocked>;
```

Use it when valid operations depend on "what mode am I in": connections, parsers, protocol handlers. Explicit transitions make illegal states unrepresentable and testable.

Other patterns worth recognising

PATTERN	IDEA	WHERE YOU ALREADY SEE IT IN C++
RAII	object owns a resource and releases it in its destructor	`vector`, `unique_ptr`, `lock_guard`
Adapter	wrap an interface so it matches the one clients expect	`std::stack` (adapts `deque`), wrapping a C library
Decorator	wrap an object to add behaviour while keeping its interface	buffered/compressed stream wrappers, logging wrappers
Proxy	a stand-in that controls access to another object	`shared_ptr`, `vector<bool>::reference`

Command	turn a request into an object (queue it, undo it)	task queues of `std::function`, thread pools
Iterator	traverse a collection without exposing its structure	the entire STL
Visitor	add operations to a type hierarchy without changing it	`std::visit` with `std::variant`
Builder	construct a complex object step by step	fluent config objects: `Req().url(u).timeout(5).build()`
PIMPL	hide implementation behind a pointer to keep headers stable and compile times low	library public headers
CRTP	a class derives from a template instantiated with itself: static polymorphism	mix-ins, `std::enable_shared_from_this`

```
// CRTP: compile-time polymorphism, no vtable
template <typename Derived>
struct Shape {
    double area() const { return static_cast<const Derived*>(this)-
>areaImpl(); }
};
struct Circle : Shape<Circle> {
    double r = 1;
    double areaImpl() const { return 3.14159 * r * r; }
};
```

WITH REAL CODE Command, Decorator, Adapter, Builder, Visitor with `std::variant` and a session state machine are in More Patterns.

READ MORE

Effective Modern C++ (Meyers)

- Item 22, Pimpl: define special member functions in the implementation file (PDF pages 165 to 174)

Cheat sheet

▼ How do you make a Singleton thread-safe in C++?

Use a function-local static (Meyers Singleton). Since C++11 its initialization is guaranteed thread-safe and happens exactly once. Alternatives are `std::call_once` with an `std::once_flag`, or creating it before threads start. Hand-written double-checked locking is easy to get wrong. Remember the object's own methods still need locking if they mutate shared state.

▼ What is wrong with Singleton?

It is global mutable state with a nice name: hidden dependencies, tight coupling, hard to test or mock, awkward lifetime and destruction order, and a problem for multithreading. Prefer passing a reference (dependency injection) unless a single shared instance is truly inherent.

▼ Strategy vs State?

Both delegate behaviour to a swappable object. Strategy: the client selects an interchangeable algorithm and strategies are independent. State: the context changes its behaviour as its own state changes, and transitions are driven by the states or context.

▼ What problems can Observer cause?

Dangling observers (destroyed but still registered), modification of the observer list during notification, unexpected notification order, cascading updates and performance issues, and thread-safety issues. Fixes: RAII detach, `weak_ptr`, iterate over a copy, and do not hold locks while calling out.

▼ Why return `unique_ptr` from a factory?

It expresses that the caller takes sole ownership, prevents leaks, and the caller can convert it to a `shared_ptr` when sharing is needed. Returning a raw owning pointer is ambiguous and error-prone.

▼ How do you replace a virtual-function Strategy in modern C++?

With a callable: store a `std::function` (runtime flexibility) or take a template parameter (zero overhead, compile-time choice). A lambda becomes the strategy, and no class hierarchy is needed.

▼ What is the Open/Closed principle?

Software entities should be open to extension but closed to modification: you add new behaviour (a new strategy, a new shape type registered in a factory) without editing the existing, tested code.